Controlled vs Uncontrolled Forms in React

This is one of the MOST important React concepts.

If you deeply understand this:

- forms become easy
- React state becomes easy
- React rendering becomes easy
- React architecture becomes easier

First Understand the Core Difference

In HTML

The browser itself controls form inputs.

Example:

```
<input type="text" />
```

Browser internally stores:

- current value
- cursor position
- typing state

The DOM manages everything.

This means:

```
DOM = Source of Truth
```

In React

React can also control the input.

Instead of browser storing value:

- React state stores value

```
React State = Source of Truth
```

This creates:

Controlled Components

THE ENTIRE DIFFERENCE

Controlled	Uncontrolled
React controls input	DOM controls input
State stores value	Input itself stores value
Uses useState	Uses refs
Rerenders on typing	No rerender needed
More React way	More traditional DOM way

PART 1 — UNCONTROLLED COMPONENTS

What Is Uncontrolled?

The DOM itself manages the input.

Example:

```
function App() {
  return <input type="text" />;
}
```

Here:

- React renders input
- browser manages typing

React does NOT know:

- current value
- what user typed
- input changes

Internal Browser Behavior

When user types:

A

Browser updates DOM internally.

React is NOT involved.

Visual Representation

```
User Typing
     ↓
DOM Updates Itself
     ↓
```

React Knows Nothing

Getting Value from Uncontrolled Input

Since React doesn't track value,
we use:

refs

Example

```
import React, { useRef } from "react";

function App() {
  const inputRef = useRef();

  function handleSubmit() {
    console.log(inputRef.current.value);
  }

  return (
    <>
      <input type="text" ref={inputRef} />

      <button onClick={handleSubmit}>
        Submit
      </button>
    </>
  );
}
```

Deep Understanding of useRef

What is useRef?

```
const inputRef = useRef();
```

Creates:

```
{
   current: undefined
}
```

After render:

```
{
   current: <input DOM node>
}
```

React Attaches Real DOM Element

```
ref={inputRef}
```

React internally does:

```
inputRef.current = actualDOMNode;
```

Now you can access:

```
inputRef.current.value
```

VERY IMPORTANT

In uncontrolled components:

```
Value lives inside DOM
```

NOT in React state.

Flow of Uncontrolled Component

```
User types
     ↓
Browser updates DOM
     ↓
React does nothing
     ↓
When needed:
React reads value using ref
```

Why Called "Uncontrolled"?

Because React does NOT control the input.

The input controls itself.

PART 2 — CONTROLLED COMPONENTS

What Is Controlled Component?

React state controls the input.

Example:

```
import React, { useState } from "react";

function App() {
  const [name, setName] = useState("");

  return (
    <input
      value={name}
      onChange={(e) => setName(e.target.value)}
    />
  );
}
```

THE MOST IMPORTANT LINE

```
value={name}
```

This means:

```
Input value comes FROM React state
```

NOT from DOM.

Deep Flow of Controlled Component

Step 1 — Initial Render

State:

```
name = ""
```

React renders:

```
<input value="" />
```

Step 2 — User Types "A"

Browser temporarily updates DOM.

But immediately:

```
onChange={(e) => setName(e.target.value)}
```

runs.

Step 3 — React State Updates

```
setName("A")
```

State becomes:

```
name = "A"
```

Step 4 — React Rerenders

React rerenders component.

New render:

```
<input value="A" />
```

Step 5 — React Forces DOM Update

React updates DOM using state value.

Full Flow

```
User types
    ↓
onChange fires
    ↓
State updates
    ↓
React rerenders
    ↓
New value passed to input
    ↓
DOM updated
```

HUGE CONCEPT

In controlled components:

```
React state is the SINGLE SOURCE OF TRUTH
```

Source of Truth Meaning

Where does REAL value live?

Controlled:

```
React state
```

Uncontrolled:

```
DOM
```

Why React Loves Controlled Components

Because React can:

- validate instantly
- disable button
- show errors
- manipulate data
- format text
- sync UI
- control everything

Example — Live Validation

```
const [email, setEmail] = useState("");

return (
  <>
    <input
      value={email}
      onChange={(e) => setEmail(e.target.value)}
    />

    {email.length < 5 && <p>Too short</p>}
  </>
);
```

Impossible without controlled behavior.

Why Controlled Components Cause Rerenders

When state changes:

```
setName("Akshat")
```

React rerenders component.

Because:

- state changed
- UI depends on state

IMPORTANT PERFORMANCE CONCEPT

Every keystroke causes rerender.

```
A → rerender
Ak → rerender
Aks → rerender
```

Does Entire Page Rerender?

NO.

React:

- creates virtual DOM
- compares changes
- updates only changed parts

Why React Hook Form Became Popular

Large forms:

- many rerenders
- slower performance

React Hook Form uses:

- uncontrolled inputs internally

for performance optimization.

Controlled vs Uncontrolled — Internal Architecture

Uncontrolled Architecture

```
User
 ↓
DOM stores value
 ↓
React accesses value later using ref
```

Controlled Architecture

```
User
 ↓
Event generated
 ↓
React state updates
 ↓
React rerenders
 ↓
DOM updated from React
```

KEY DIFFERENCE

Uncontrolled

```
DOM → owns state
```

Controlled

```
React → owns state
```

VERY IMPORTANT WARNING

NEVER DO THIS

```
<input value="hello" />
```

Without `onChange`.

Why?

React says:

```
Value is controlled
```

But user cannot change it.

Input becomes readonly.

Correct Version

```
<input
  value={value}
  onChange={(e) => setValue(e.target.value)}
/>
```

---

defaultValue vs value

---

value

Controlled input.

```
<input value={name} />
```

React fully controls value.

---

defaultValue

Uncontrolled initial value.

```
<input defaultValue="Akshat" />
```

Only initial value set.

After that:
DOM controls it.

Example

```
<input defaultValue="Hello" />
```

Browser internally stores changes.

React stops controlling after initial render.

Why Uncontrolled Components Exist

Useful when:

- simple forms
- file inputs
- performance optimization
- integrating non-React libraries

VERY IMPORTANT — File Inputs

File inputs are ALWAYS uncontrolled.

```
<input type="file" />
```

Browser security restrictions prevent React control.

Use refs.

Example

```
const fileRef = useRef();

function handleUpload() {
  console.log(fileRef.current.files);
}
```

Mental Model

Controlled Input

React says:

```
"I own this value."
```

Uncontrolled Input

DOM says:

```
"I own this value."
```

Real-World Analogy

Controlled

Teacher controls everything.

```
Student speaks
 ↓
Teacher approves
 ↓
Teacher writes final answer
```

Uncontrolled

Student writes independently.

Teacher checks later.

Advanced Concept — Hybrid Forms

Some libraries mix both approaches.

Example:

- uncontrolled typing
- controlled validation

This improves performance.

React Internally Prefers Predictability

Controlled components:

- predictable
- debuggable
- easier state synchronization

That is why they are recommended.

When to Use Controlled Components

Use when:

- validation needed
- dynamic UI
- conditional rendering
- live updates

- API synchronization
- dependent fields

Most professional apps use controlled forms.

When to Use Uncontrolled Components

Use when:

- simple forms
- performance critical forms
- file uploads
- third-party integrations

Interview-Level Difference

Controlled	Uncontrolled
State managed by React	State managed by DOM
Uses useState	Uses ref
Predictable	Less predictable
Easy validation	Harder validation
More rerenders	Better performance
Recommended	Less common

Ultimate Deep Understanding

Controlled Components

```
DOM becomes a reflection of React state
```

Uncontrolled Components

```
React reads data FROM DOM when needed
```

MOST IMPORTANT LINE OF THE ENTIRE TOPIC

Controlled components:

```
React controls the DOM
```

Uncontrolled components:

```
DOM controls itself
```